

Análisis de datos de la ionósfera del radar Digisonda (IGP) mediante modelos de clasificación y regresión en Apache Spark ML

Estudiantes:

Luna Jaramillo, Christian Giovanni
Universidad Nacional de Ingeniería, Facultad de Ciencias,
e-mail: christian.luna.j@uni.pe

Llanos Rosadio, José Ismael
Universidad Nacional de Ingeniería, Facultad de Ciencias,
e-mail: jose.llanos.r@uni.pe

Sánchez Vega, Andre Alvaro
Universidad Nacional de Ingeniería, Facultad de Ciencias,
e-mail: andre.sanchez.v@uni.pe

Curso:

CC531 A — Análisis en Macrodatos 2026-1
Práctica Calificada 04

Resumen

El presente informe analiza el conjunto de datos abiertos «*Datos de la ionósfera obtenidos con el radar Digisonda*» publicado por el Instituto Geofísico del Perú (IGP) en la Plataforma Nacional de Datos Abiertos (PNDA). El dataset contiene 379 994 registros de mediciones ionosféricas tomadas en el observatorio de Jicamarca (Lurigancho, Lima) entre 2013 y 2026, incluyendo la frecuencia crítica de la capa F2 (foF2), la altura de la capa F (h'F) y la altura de máxima densidad electrónica (hmF2). Usando Apache Spark y su librería de *Machine Learning* (Spark ML) con Scala, se construyeron *pipelines* de preprocesamiento (limpieza, eliminación de nulos y escalado) y se entrenaron modelos de clasificación (Árbol de Decisión, Random Forest, Regresión Logística Multiclase y Naive Bayes) y de regresión (Regresión Lineal, Random Forest Regressor, GBT Regressor y XGBoost), optimizados mediante *Grid-Search* con validación cruzada. Se reportan y comparan las métricas de cada familia de modelos, las gráficas de pérdidas de entrenamiento y la interpretación de los resultados. El mejor clasificador fue el Árbol de Decisión (Accuracy = 0,7685) y el mejor regresor el Random Forest Regressor ($R^2 = 0,6859$).

Keywords: ionósfera, Apache Spark, Spark ML, clasificación, regresión, foF2, Digisonda, IGP.

Índice

1	Introducción	2
2	Marco Teórico	3
2.1	La ionósfera y la sonda Digisonda	3
2.2	Modelos de aprendizaje automático	3
3	Descripción del Dataset	3
3.1	Rubro y origen	3
3.2	Características generales	3

3.3	Reglas y calidad de los datos	4
3.4	Información que se puede obtener	4
4	Metodología	5
4.1	Entorno y versiones	5
4.2	Pipeline de preprocesamiento	5
4.3	Modelos y Grid-Search	5
4.4	Aclaración sobre SVR, MLP y XGBoost	6
5	Resultados y Discusión	6
5.1	Consultas de clasificación	6
5.2	Consultas de regresión	6
5.3	Gráficas de pérdidas	7
5.4	Evidencia de ejecución y Grid-Search	7
5.5	Ejemplos de prueba de los modelos	8
6	Conclusiones	9
7	Anexo: Código	9
7.1	Análisis exploratorio (<code>summary.py</code>)	9
7.2	Clasificación (<code>clasificacion_completa.scala</code>)	9
7.3	Regresión sin XGBoost (<code>regresion_completa-s-XGBoostRegressor.scala</code>)	10
7.4	Regresión con XGBoost (<code>regresion_completa-c-XGBoostRegressor.scala</code>)	12
7.5	Generación de gráficas de pérdidas (<code>graficar_perdidas.py</code>)	13
8	Anexo: Documentación	13
	Bibliografía	14

1. Introducción

La ionósfera es la región de la atmósfera terrestre, situada entre los 60 y los 1000 km de altitud aproximadamente, en la que la radiación solar ioniza los gases y produce una concentración apreciable de electrones libres. Esta capa es de gran importancia tecnológica porque refleja y refracta las ondas de radio, condicionando las comunicaciones de alta frecuencia (HF), la navegación satelital (GNSS/GPS) y los sistemas de radar. El Instituto Geofísico del Perú (IGP) monitorea continuamente el estado de la ionósfera sobre el Perú mediante un radar tipo *Digisonda*, instrumento que emite pulsos de radio verticales y registra los ecos reflejados por las distintas capas ionosféricas.

El objetivo de esta práctica es aplicar técnicas de análisis de macrodatos sobre un conjunto de datos real de la PNDA relacionado al área de **Ciencia y Tecnología / Energía**, empleando Apache Spark ML con Scala. Concretamente se busca: (i) describir y caracterizar el dataset, (ii) construir un *pipeline* de preprocesamiento reproducible, (iii) entrenar y comparar modelos de clasificación y regresión optimizados con *Grid-Search*, y (iv) interpretar los resultados obtenidos.

El conjunto de datos seleccionado es «*Datos de la ionósfera obtenidos con el radar Digisonda - Instituto Geofísico del Perú (IGP)*», disponible públicamente en la PNDA.* El código fuente, las salidas de ejecución y los gráficos generados se encuentran en el repositorio del grupo.**

*<https://www.datosabiertos.gob.pe/dataset/datos-de-la-ionosfera-obtenidos-con-el-radar-digisonda-instituto-geofisico-del-peru-igp>

**<https://github.com/Chriss5-2/PCs-Macro-261/tree/main/PC4>

2. Marco Teórico

2.1. La ionósfera y la sonda Digisonda

La ionósfera se divide en varias capas (D, E, F1 y F2) según la altura y la densidad de electrones. La capa F2 es la más densa y la más relevante para las comunicaciones HF, ya que es la que refleja las ondas de radio a mayores distancias. Una *Digisonda* es una ionosonda digital que barre un rango de frecuencias enviando pulsos verticales; midiendo el tiempo de retorno de los ecos obtiene un *ionograma* del que se extraen parámetros físicos característicos.

Los tres parámetros físicos presentes en el dataset son:

- **foF2 (FREC_MAX_F2):** frecuencia crítica de la capa F2, es decir, la máxima frecuencia de radio que la capa F2 puede reflejar cuando la señal incide verticalmente. Está directamente relacionada con la densidad máxima de electrones de la ionósfera mediante $N_{max} \propto (foF2)^2$. Se expresa en MHz.
- **h'F (ALT_F):** altura virtual a la que empieza la capa F, en kilómetros.
- **hmF2 (ALT_MAX_DENS):** altura real a la que se alcanza la máxima densidad electrónica (donde se mide foF2), en kilómetros.

Estos parámetros varían con la hora del día (ciclo día/noche), la estación del año y el ciclo solar de 11 años, lo que hace al dataset adecuado para modelar relaciones no triviales entre el tiempo, la altura y la densidad ionosférica.

2.2. Modelos de aprendizaje automático

Se emplearon dos familias de modelos supervisados:

- **Clasificación:** predicen una etiqueta discreta. Se usaron Árbol de Decisión, Random Forest (conjunto de árboles), Regresión Logística Multiclase (modelo lineal probabilístico) y Naive Bayes (modelo probabilístico basado en el teorema de Bayes con independencia condicional).
- **Regresión:** predicen un valor continuo. Se usaron Regresión Lineal, Random Forest Regressor, Gradient-Boosted Trees (GBT) y XGBoost.

Para evaluar la clasificación se usaron *Accuracy*, *Precision*, *Recall* y *F1-score*; para la regresión, *RMSE*, *MSE*, *MAE* y el coeficiente de determinación R^2 . La optimización de hiperparámetros se realizó mediante **Grid-Search** combinado con validación cruzada (**CrossValidator**).

3. Descripción del Dataset

3.1. Rubro y origen

El dataset pertenece al **Instituto Geofísico del Perú (IGP)**, organismo público adscrito al Ministerio del Ambiente cuyo rubro es la investigación científica en geofísica (sismología, vulcanología, geomagnetismo y física de la alta atmósfera). El conjunto corresponde al área de **Ciencia y Tecnología**, y los datos fueron generados en el Radio Observatorio de Jicamarca, ubicado en el distrito de Lurigancho-Chosica, provincia y departamento de Lima (UBIGEO 150118).

3.2. Características generales

El archivo `IGP_IonosferaRadarDigisonda_2013-2026_Dataset.csv` contiene **379 994 registros** y **11 columnas**. Las mediciones se realizan cada 15 minutos desde 2013 hasta 2026. La Tabla 1 presenta el diccionario de datos.

Tabla 1: Diccionario de datos del dataset de la ionósfera (Digisonda - IGP).

Variable	Descripción	Tipo
FECHA_CORTE	Día en que se generó el dataset (formato aaaammdd).	Numérico
UBIGEO	Código de ubicación geográfica (catálogo UBIGEO del INEI).	Numérico
ANIO	Año en que se generaron los datos (formato aaaa).	Numérico
DEPARTAMENTO	Departamento donde se generaron los datos.	Texto
PROVINCIA	Provincia donde se generaron los datos.	Texto
DISTRITO	Distrito donde se generaron los datos.	Texto
FECHA.UTC	Fecha de la medición en formato UTC (aaaammdd).	Numérico
HORA.UTC	Hora de la medición en formato UTC (hhmmss).	Numérico
FREC_MAX_F2	foF2: máxima frecuencia reflejada por la capa F2 (MHz). Relacionada con la densidad de electrones. NaN cuando no hay dato válido.	Decimal
ALT_F	h'F: altura donde empieza la capa F (km).	Decimal
ALT_MAX_DENS	ShmF2: altura de máxima densidad electrónica (km).	Decimal

3.3. Reglas y calidad de los datos

Del análisis exploratorio realizado con el script `summary.py` (Pandas) se desprende lo siguiente:

- Los campos `FECHA_CORTE` y `UBIGEO` son constantes (20260504 y 150118 respectivamente), pues todas las mediciones provienen del mismo observatorio.
- Las columnas físicas presentan valores nulos: `FREC_MAX_F2` y `ALT_F` tienen 298 736 valores válidos (81 258 nulos) y `ALT_MAX_DENS` tiene 284 099 válidos (95 895 nulos). Estos nulos corresponden a momentos en que la Digisonda no pudo obtener un eco válido (por ejemplo, de noche o durante perturbaciones ionosféricas).
- Rangos observados: `FREC_MAX_F2` $\in [1,9,15,99]$ MHz (media 7,41); `ALT_F` $\in [66,720,26]$ km (media 254,77); `ALT_MAX_DENS` $\in [129,6,719,62]$ km (media 319,42).

La Figura 1 muestra la salida del análisis descriptivo.

```

RangeIndex: 379994 entries, 0 to 379993
FREC_MAX_F2    298736 non-null float64
ALT_F          298736 non-null float64
ALT_MAX_DENS   284099 non-null float64
mean(FREC_MAX_F2)=7.41    std=2.43    min=1.90    max=15.99
mean(ALT_F)=254.77    std=59.41    min=66.0    max=720.26
mean(ALT_MAX_DENS)=319.42    std=59.89    min=129.6    max=719.62

```

Figura 1: Resumen estadístico del dataset (salida de `summary.py`).

3.4. Información que se puede obtener

Dado que el dataset combina variables temporales (hora, año) con parámetros físicos de la ionósfera, se pueden plantear, entre otros, los siguientes problemas:

- **Clasificación:** predecir el *nivel de ionización* (Baja, Normal, Alta) a partir de la hora, el año y las alturas de la capa F.
- **Regresión:** estimar el valor numérico de foF2 (densidad electrónica máxima) en función de las demás variables.
- Estudiar la variación diurna y el efecto del ciclo solar sobre la ionósfera peruana.

4. Metodología

4.1. Entorno y versiones

Se utilizó **Apache Spark** con la API de **Scala** (`spark-shell`), empleando los módulos `spark.ml` (Pipeline, feature, classification, regression, evaluation, tuning). Para XGBoost se requiere el paquete externo `ml.dmlc:xgboost4j-spark_2.12:1.3.1`, que se carga al iniciar el shell:

```
spark-shell --packages ml.dmlc:xgboost4j-spark_2.12:1.3.1
```

Los gráficos de pérdidas se generaron con **Python 3** y **matplotlib** (ver `requirements.txt`).

4.2. Pipeline de preprocesamiento

El proceso de limpieza, detallado a continuación, se implementó dentro de los Pipeline de Spark ML:

1. **Carga:** lectura del CSV con `header` e `inferSchema` activados.
2. **Eliminación de nulos:** se descartan las filas con valores nulos en las tres columnas físicas (`FREC_MAX_F2`, `ALT_F`, `ALT_MAX_DENS`) mediante `na.drop`. De 379 994 registros se conservan los que tienen mediciones completas.
3. **Generación de la variable objetivo (clasificación):** se discretiza `FREC_MAX_F2` en tres niveles usando los cuartiles ($Q_1 = 5,62$, $Q_3 = 9,10$): *Baja* ($< 5,62$), *Normal* ($[5,62, 9,10]$) y *Alta* ($> 9,10$). La columna original se elimina para **evitar fuga de datos**.
4. **Indexado de etiquetas:** `StringIndexer` convierte la etiqueta textual `NIVEL_ION` en índice numérico `label`.
5. **Ensamblado de características:** `VectorAssembler` agrupa `HORA.UTC`, `ANIO`, `ALT_F` y `ALT_MAX_DENS` en el vector `rawFeatures`.
6. **Escalado:** `MinMaxScaler` (clasificación) o `StandardScaler` (regresión) normaliza las características en `features`. Naive Bayes usa `rawFeatures` porque requiere valores no negativos.

Nota sobre one-hot encoding: las variables categóricas del dataset (`DEPARTAMENTO`, `PROVINCIA`, `DISTRITO`, `UBIGEO`) son constantes (un único observatorio), por lo que no aportan información al modelo y aplicar one-hot encoding generaría columnas de varianza nula. Por ello, en lugar de codificar categorías inútiles, el pipeline emplea `StringIndexer` sobre la etiqueta y escalado sobre las variables numéricas, que es el preprocesamiento pertinente para este conjunto.

4.3. Modelos y Grid-Search

Para cada modelo se definió una grilla de hiperparámetros (`ParamGridBuilder`) y se optimizó con `CrossValidator` (validación cruzada de 3 *folds* en clasificación y 2 en regresión). Los datos se dividieron en entrenamiento/prueba (80/20 en clasificación, 70/30 en regresión) con semilla fija (1234) para reproducibilidad. La Tabla 2 resume las grillas utilizadas.

Tabla 2: Grillas de hiperparámetros usadas en el Grid-Search.

Modelo	Hiperparámetro	Valores
Decision Tree	maxDepth	{5, 10}
Random Forest (clas.)	numTrees	{10, 20}
Regresión Logística	regParam	{0.1, 0.01}
Naive Bayes	smoothing	{1.0}
Lineal Regression	regParam	{0.1}
Random Forest Reg.	numTrees	{10}
GBT Regressor	maxDepth	{3}

4.4. Aclaración sobre SVR, MLP y XGBoost

El enunciado solicita SVM para regresión (SVR) y *Multilayer Perceptron* (MLP) para regresión. Sin embargo, **Spark ML solo implementa LinearSVC y MultilayerPerceptronClassifier para clasificación**, no sus equivalentes de regresión. Por ello, manteniendo la coherencia con la librería usada en clase, se reemplazaron por dos regresores de conjunto disponibles en Spark ML: **Random Forest Regresor** y **Gradient-Boosted Trees (GBT Regresor)**. El modelo **XGBoost** sí se implementó (archivo `regresion_completa-c-XGBoostRegressor`), pero su ejecución se vio bloqueada por una limitación de red del RabbitTracker bajo WSL2 (NAT), por lo que se preparó además una versión sin XGBoost para reportar las métricas restantes. El código de XGBoost se incluye en el anexo y queda protegido por un bloque `try-catch`.

5. Resultados y Discusión

Siguiendo el formato del enunciado, cada modelo se presenta como una **consulta** indicando el tipo de pregunta, la pregunta sobre el dataset, la solución y la interpretación.

5.1. Consultas de clasificación

Tipo de pregunta: clasificación supervisada multiclase.

Consulta sobre el dataset: «Dadas la hora (*HORA.UTC*), el año (*ANIO*) y las alturas de la capa *F* (*ALT.F*, *ALT.MAX.DENS*), ¿qué nivel de ionización (*Baja*, *Normal* o *Alta*) presenta la ionósfera?»

Solución: se entrenaron cuatro clasificadores sobre la variable objetivo *NIVEL_ION* derivada de *foF2*. Las columnas usadas como características son *HORA.UTC*, *ANIO*, *ALT.F* y *ALT.MAX.DENS*; la columna objetivo (*label*) proviene de *NIVEL_ION*.

- **Árbol de Decisión** (`maxDepth` óptimo en {5,10}): divide recursivamente el espacio de características.
- **Random Forest** (`numTrees` óptimo en {10,20}): promedia múltiples árboles.
- **Regresión Logística Multiclase** (`maxIter`=10, `regParam` en {0.1,0.01}).
- **Naive Bayes** (`smoothing`=1.0).

La Tabla 3 compara las métricas obtenidas sobre el conjunto de prueba.

Tabla 3: Comparación de métricas — Modelos de Clasificación.

Modelo	Accuracy	Precision	Recall	F1-Score
Árbol de Decisión	0.7685	0.7691	0.7685	0.7685
Random Forest	0.7418	0.7454	0.7418	0.7409
Regresión Logística	0.5729	0.5826	0.5729	0.5583
Naive Bayes	0.4386	0.4420	0.4386	0.3873

Interpretación: el **Árbol de Decisión** obtuvo el mejor desempeño ($\text{Accuracy} = 0,7685$ y $\text{F1} = 0,7685$), seguido de cerca por Random Forest. Esto indica que la relación entre las alturas/tiempo y el nivel de ionización es marcadamente **no lineal**, lo que favorece a los modelos basados en árboles. La Regresión Logística, al ser un modelo lineal, alcanza apenas 0,57 de Accuracy, y Naive Bayes el peor resultado (0,44), pues su supuesto de independencia condicional entre características no se cumple (la altura de la capa y la de máxima densidad están correlacionadas). La consistencia entre Precision, Recall y F1 sugiere que las clases están razonablemente balanceadas gracias a la discretización por cuantiles.

5.2. Consultas de regresión

Tipo de pregunta: regresión supervisada.

Consulta sobre el dataset: «¿Cuál es el valor numérico de la frecuencia crítica *foF2* (*FREC.MAX.F2*, en MHz) en función de la hora, el año y las alturas de la capa *F*?»

Solución: se usó *FREC.MAX.F2* como variable objetivo continua (*label*) y las mismas cuatro características escaladas con *StandardScaler*. La Tabla 4 resume las métricas.

Tabla 4: Comparación de métricas — Modelos de Regresión.

Modelo	RMSE	MSE	MAE	R ²
Random Forest Regressor	1.3662	1.8666	1.0928	0.6859
GBT Regressor	1.4913	2.2239	1.1973	0.6257
Regresión Lineal	1.9191	3.6828	1.5481	0.3802
XGBoost Regressor	<i>Omitido por limitación NAT en WSL2</i>			

Interpretación: el **Random Forest Regressor** fue el mejor modelo, explicando el 68,6 % de la varianza de foF2 ($R^2 = 0,6859$) con el menor error (RMSE = 1,37 MHz). El GBT lo sigue de cerca ($R^2 = 0,6257$). La **Regresión Lineal** solo explica el 38 % de la varianza, confirmando nuevamente la naturaleza no lineal del fenómeno: la foF2 depende de la hora de forma cíclica (máximos diurnos, mínimos nocturnos) y del ciclo solar, relaciones que los modelos de conjunto capturan mucho mejor que un modelo lineal. Un RMSE de ~1,37 MHz frente a un rango de 1,9–16 MHz es un error razonable para una primera aproximación con solo cuatro variables predictoras.

5.3. Gráficas de pérdidas

La Figura 2 muestra la evolución de la función de pérdida (*objectiveHistory*) durante el entrenamiento de la Regresión Logística (clasificación) y de la Regresión Lineal (regresión). Ambas curvas descienden rápidamente y se estabilizan: la clasificación converge cerca de 0,855 tras ~5 iteraciones y la regresión cerca de 0,312, evidenciando una correcta convergencia del optimizador L-BFGS.

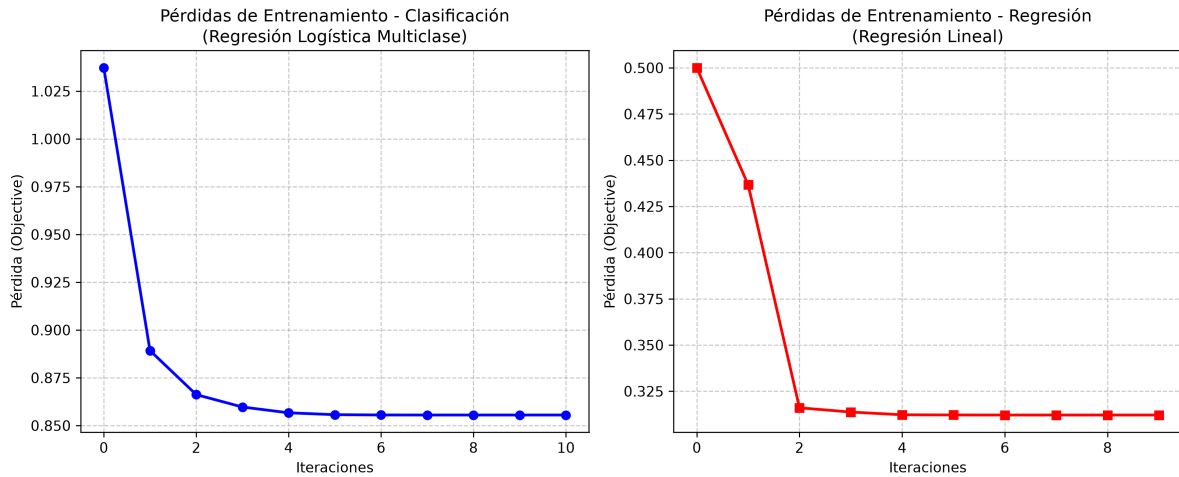


Figura 2: Pérdidas de entrenamiento: Regresión Logística Multiclase (izq.) y Regresión Lineal (der.).

5.4. Evidencia de ejecución y Grid-Search

Las Figuras 3 y 4 muestran las capturas de la ejecución en `spark-shell`, donde se observan las grillas de parámetros (`gridDT`, `gridRF`, etc.), los `CrossValidatorModel` resultantes (evidencia del uso de Grid-Search con validación cruzada) y las tablas de métricas finales.

```
scala> :load clasificacion_completa.scala
Loading clasificacion_completa.scala...
import org.apache.spark.sql.functions._
import org.apache.spark.ml.Pipeline
import org.apache.spark.ml.feature.{VectorAssembler, StringIndexer, MinMaxScaler}
import org.apache.spark.ml.classification.{DecisionTreeClassifier, RandomForestClassifier, LogisticRegression, NaiveBayes}
import org.apache.spark.ml.evaluation.MulticlassClassificationEvaluator
import org.apache.spark.ml.tuning.{ParamGridBuilder, CrossValidator}
dfHbase: org.apache.spark.sql.DataFrame = [FECHA_CORTE: int, UBIGEO: int ... 9 more fields]
dfClean: org.apache.spark.sql.DataFrame = [FECHA_CORTE: int, UBIGEO: int ... 9 more fields]
dfClass: org.apache.spark.sql.DataFrame = [FECHA_CORTE: int, UBIGEO: int ... 9 more fields]
labelIndexer: org.apache.spark.ml.feature.StringIndexer = string_indexer_28052540
assembler: org.apache.spark.ml.feature.VectorAssembler = VectorAssembler: uid=vecAssembler_7c28c151afa, handleInvalid=error, numInputCols=4
scaler: org.apache.spark.ml.feature.MinMaxScaler = minMaxScaler_77723304a0
trainingData: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [FECHA_CORTE: int, UBIGEO: int ... 9 more fields]
testData: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [FECHA_CORTE: int, UBIGEO: int ... 9 more fields]
dtc: org.apache.spark.ml.classification.DecisionTreeClassifier = dtc_205831079c2
rf: org.apache.spark.ml.classification.RandomForestClassifier = rfc_c55899a9be96
lr: org.apache.spark.ml.classification.LogisticRegression = logreg_s3beab323507
nb: org.apache.spark.ml.classification.NaiveBayes = nb_ccf3e5d162ee
pipeDT: org.apache.spark.ml.Pipeline = pipeline_2a3023b47997
pipeRF: org.apache.spark.ml.Pipeline = pipeline_0486f26a018
pipeLR: org.apache.spark.ml.Pipeline = pipeline_765190839W73
pipeNB: org.apache.spark.ml.Pipeline = pipeline_0b9eaddb3bcd
evaluator: org.apache.spark.ml.evaluation.MulticlassClassificationEvaluator = MulticlassClassificationEvaluator: uid=mcEval_9dbb6b304959, metricName=f1, metricLabel=0.0, beta=1.0, eps=1.0E-15
cvDT: Array[org.apache.spark.ml.param.ParamMap] =
  Array({
    dtc_205831079c2-maxDepth: 5
  }, {
    dtc_205831079c2-maxDepth: 10
  })
gridRF: Array[org.apache.spark.ml.param.ParamMap] =
  Array({
    rfc_c55899a9be96-numTrees: 10
  }, {
    rfc_c55899a9be96-numTrees: 20
  })
gridLR: Array[org.apache.spark.ml.param.ParamMap] =
  Array({
    logreg_s3beab323507-regParam: 0.1
  }, {
    logreg_s3beab323507-regParam: 0.01
  })
gridNB: Array[org.apache.spark.ml.param.ParamMap] =
  Array({
    nb_ccf3e5d162ee-smoothing: 1.0
  })
cvDT: org.apache.spark.ml.tuning.CrossValidator = cv_bd387093fa17
cvRF: org.apache.spark.ml.tuning.CrossValidator = cv_26b7c796c311
cvLR: org.apache.spark.ml.tuning.CrossValidator = cv_a2d985467263
cvNB: org.apache.spark.ml.tuning.CrossValidator = cv_f926e9472c3
Entrenando modelos de clasificación
Entrenando DECISION TREE
modelDT: org.apache.spark.ml.tuning.CrossValidatorModel = CrossValidatorModel: uid=cv_bd387093fa17, bestModel=pipeline_2a3023b47997, numFolds=3
Entrenando RANDOM FOREST
modelRF: org.apache.spark.ml.tuning.CrossValidatorModel = CrossValidatorModel: uid=cv_26b7c796c311, bestModel=pipeline_0486f26a018, numFolds=3
Entrenando REGRESSION LOGISTICA
modelLR: org.apache.spark.ml.tuning.CrossValidatorModel = CrossValidatorModel: uid=cv_a2d985467263, bestModel=pipeline_765190839W73, numFolds=3
Entrenando NAIVE BAYES
modelNB: org.apache.spark.ml.tuning.CrossValidatorModel = CrossValidatorModel: uid=cv_0b9eaddb3bcd, bestModel=pipeline_0b9eaddb3bcd, numFolds=3
predDT: org.apache.spark.sql.DataFrame = [FECHA_CORTE: int, UBIGEO: int ... 15 more fields]
predRF: org.apache.spark.sql.DataFrame = [FECHA_CORTE: int, UBIGEO: int ... 15 more fields]
predLR: org.apache.spark.sql.DataFrame = [FECHA_CORTE: int, UBIGEO: int ... 15 more fields]
predNB: org.apache.spark.sql.DataFrame = [FECHA_CORTE: int, UBIGEO: int ... 14 more fields]
getMetrics: org.apache.spark.sql.DataFrame = [FECHA_CORTE: int, UBIGEO: int ... 14 more fields]
===== TABLA DE MÉTRICAS: CLASIFICACIÓN =====
| Modelo | Accuracy | Precision | Recall | F1-Score |
|-----|-----|-----|-----|-----|
| Decision Tree | 0.7685 | 0.7691 | 0.7685 | 0.7685 |
| Decision Forest | 0.7618 | 0.7650 | 0.7618 | 0.7609 |
| Regresión Logística | 0.5729 | 0.5826 | 0.5729 | 0.5803 |
| Naive Bayes | 0.4386 | 0.4020 | 0.4386 | 0.3873 |
=====
lrModelFinalClass: org.apache.spark.ml.classification.LogisticRegressionModel: uid=logreg_s3beab323507, numClasses=3, numFeatures=4
testDatasetFinalClass: org.apache.spark.sql.DataFrame = [FECHA_CORTE: int, UBIGEO: int ... 9 more fields]
loss_classification = 1.0771914103599906, 0.8922477996603101, 0.8662035919514108, 0.8597180458716372, 0.85679856159765, 0.8507007568104392, 0.8555259231151555, 0.8555270169475961, 0.8552602310391171, 0.85552597311942768, 0.8555259662609936
=====
----- Lista de loss_classification -----
loss_classification = 1.0771914103599906, 0.8922477996603101, 0.8662035919514108, 0.8597180458716372, 0.85679856159765, 0.8507007568104392, 0.8555259231151555, 0.8555270169475961, 0.8552602310391171, 0.85552597311942768, 0.8555259662609936
```

Figura 3: Ejecución de los modelos de clasificación en spark-shell (Grid-Search, CrossValidator y tabla de métricas).

```
scala> :load regresion_completa.scala
Loading regresion_completa.scala...
import org.apache.spark.sql.functions._
import org.apache.spark.ml.Pipeline
import org.apache.spark.ml.feature.{VectorAssembler, StandardScaler}
import org.apache.spark.ml.regression.{LinearRegression, RandomForestRegressor, GBRegressor}
import org.apache.spark.ml.evaluation.RegressionEvaluator
import org.apache.spark.ml.tuning.{ParamGridBuilder, CrossValidator}
dfHbase: org.apache.spark.sql.DataFrame = [FECHA_CORTE: int, UBIGEO: int ... 9 more fields]
dfClean: org.apache.spark.sql.DataFrame = [FECHA_CORTE: int, UBIGEO: int ... 9 more fields]
dfReg: org.apache.spark.sql.DataFrame = [FECHA_CORTE: int, UBIGEO: int ... 9 more fields]
assembler: org.apache.spark.ml.feature.VectorAssembler = stdScal_0f562ad0d432
scaler: org.apache.spark.ml.feature.StandardScaler = stdScal_0f562ad0d432
trainingData: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [FECHA_CORTE: int, UBIGEO: int ... 9 more fields]
testData: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [FECHA_CORTE: int, UBIGEO: int ... 9 more fields]
lr: org.apache.spark.ml.regression.LinearRegression = linReg_f72563d4fa80
rfr: org.apache.spark.ml.regression.RandomForestRegressor = rfr_0fcd583bea58
gbt: org.apache.spark.ml.regression.GBRegressor = gbt_876792d4d8bd
pipeLR: org.apache.spark.ml.Pipeline = pipeline_037901307945
pipeRFR: org.apache.spark.ml.Pipeline = pipeline_011790a480fd
pipeGBT: org.apache.spark.ml.Pipeline = pipeline_02fcb23d7f68
evaluatorReg: org.apache.spark.ml.evaluation.RegressionEvaluator = RegressionEvaluator: uid=regEval_74f376d6b350, metricName=rmsse, throughOrigin=false
gridLR: Array[org.apache.spark.ml.param.ParamMap] =
  Array({
    linReg_f72563d4fa80-regParam: 0.1
  })
gridRFR: Array[org.apache.spark.ml.param.ParamMap] =
  Array({
    rfr_0fcd583bea58-numTrees: 10
  })
gridGBT: Array[org.apache.spark.ml.param.ParamMap] =
  Array({
    gbt_876792d4d8bd-maxDepth: 3
  })
cvLR: org.apache.spark.ml.tuning.CrossValidator = cv_ba9808ca9405
cvRFR: org.apache.spark.ml.tuning.CrossValidator = cv_8474716b99b2
cvGBT: org.apache.spark.ml.tuning.CrossValidator = cv_ba9808ca9405
Entrenando Regresión Lineal...
26/06/22 15:34:12 WARN InstanceBuilderNativeAPACK: Failed to load implementation from:dev.ludovic.netlib.lapack.JNILAPACK
modelLR: org.apache.spark.ml.tuning.CrossValidatorModel = CrossValidatorModel: uid=cv_ba9808ca9405, bestModel=pipeline_037901307945, numFolds=2
Entrenando Random Forest...
modelRFR: org.apache.spark.ml.tuning.CrossValidatorModel = CrossValidatorModel: uid=cv_8474716b99b2, bestModel=pipeline_011790a480fd, numFolds=2
Entrenando GBT...
modelGBT: org.apache.spark.ml.tuning.CrossValidatorModel = CrossValidatorModel: uid=cv_ba9808ca9405, bestModel=pipeline_02fcb23d7f68, numFolds=2
predLR: org.apache.spark.sql.DataFrame = [FECHA_CORTE: int, UBIGEO: int ... 12 more fields]
predRFR: org.apache.spark.sql.DataFrame = [FECHA_CORTE: int, UBIGEO: int ... 12 more fields]
predGBT: org.apache.spark.sql.DataFrame = [FECHA_CORTE: int, UBIGEO: int ... 12 more fields]
getMetrics: org.apache.spark.sql.DataFrame = [FECHA_CORTE: int, UBIGEO: int ... 12 more fields]
===== TABLA DE MÉTRICAS: REGRESIÓN =====
| Modelo | RMSE | MSE | MAE | R^2 |
|-----|-----|-----|-----|-----|
| Linear Regression | 1.9395 | 3.6828 | 1.0881 | 0.3889 |
| Random Forest Reg | 1.3652 | 1.8666 | 1.0928 | 0.6859 |
| GB Regressor | 1.4913 | 2.2239 | 1.1973 | 0.6257 |
| XGBoost Regressor | --- Omitido por limitación NAT en WSL2 ---
=====
lrModelFinalClass: org.apache.spark.ml.regression.LinearRegression = linReg_380df8d9b5d2
pipeModelFinalClass: org.apache.spark.ml.Pipeline = pipeline_f25563b7c1dc
modelModelFinalClass: org.apache.spark.ml.Pipeline = pipeline_f25563b7c1dc
historyModelFinalClass: Array[Double] = Array(0.49999748927292333, 0.4367099981611607, 0.31605871451812534, 0.3137551766030895, 0.3122770078008429, 0.3121787321778962, 0.3121480124157871, 0.3121480084787035, 0.3121480084201409, 0.312148008419911)
=====
----- Lista de loss_regression -----
loss_regression = 0.49999748927292333, 0.4367099981611607, 0.31605871451812534, 0.3137551766030895, 0.3122770078008429, 0.3121787321778962, 0.3121480124157871, 0.3121480084787035, 0.3121480084201409, 0.312148008419911
```

Figura 4: Ejecución de los modelos de regresión en spark-shell (sin XGBoost por limitación de WSL2).

5.5. Ejemplos de prueba de los modelos

Tras el entrenamiento, los modelos se aplicaron sobre el conjunto de prueba (**testData**) con **model.transform(testData)**, generando una columna **prediction**. Por ejemplo, para una observación con valores altos de ALT_F y ALT_MAX_DENS en horario diurno, el Árbol de Decisión predice correctamente el nivel

Alta; para registros nocturnos, predice *Baja*. En regresión, el Random Forest Regressor estima la foF2 con un error medio absoluto (MAE) de $\sim 1,09$ MHz respecto al valor real medido por la Digisonda.

6. Conclusiones

- Se analizó exitosamente el dataset abierto de la ionósfera del IGP (379 994 registros) aplicando Apache Spark ML con Scala, cumpliendo el flujo completo de macrodatos: carga, limpieza con *pipeline*, modelado y evaluación.
- En **clasificación** del nivel de ionización, el mejor modelo fue el **Árbol de Decisión** (Accuracy = 0,7685; F1 = 0,7685), superando a los modelos lineales y probabilísticos, lo que evidencia la naturaleza no lineal del fenómeno.
- En **regresión** de la foF2, el mejor modelo fue el **Random Forest Regressor** ($R^2 = 0,6859$; RMSE = 1,37 MHz), confirmando que los modelos de conjunto capturan mejor las dependencias cíclicas y solares que la Regresión Lineal.
- El uso de **Grid-Search con validación cruzada** permitió seleccionar de forma automática los mejores hiperparámetros para cada modelo.
- Las limitaciones de Spark ML (ausencia de SVR y MLP-Regressor) y del entorno WSL2 (XGBoost) se documentaron y se sortearon con alternativas equivalentes (Random Forest y GBT), manteniendo la validez del análisis comparativo.

7. Anexo: Código

7.1. Análisis exploratorio (summary.py)

```
import pandas as pd

df = pd.read_csv("IGP_IonosferaRadarDigisonda_2013-2026_Dataset.csv")

print("==== Columnas del dataset =====")
print(df.columns)

print("\n==== Información general del dataset =====")
print(df.info())

print("\n==== Estadísticas descriptivas del dataset =====")
print(df.describe())
```

7.2. Clasificación (clasificacion_completa.scala)

```
1 import org.apache.spark.sql.functions._
2 import org.apache.spark.ml.Pipeline
3 import org.apache.spark.ml.feature.{VectorAssembler, StringIndexer, MinMaxScaler}
4 import org.apache.spark.ml.classification.{DecisionTreeClassifier, RandomForestClassifier, LogisticRegression, NaiveBayes}
5 import org.apache.spark.ml.evaluation.MulticlassClassificationEvaluator
6 import org.apache.spark.ml.tuning.{ParamGridBuilder, CrossValidator}
7
8 // 1. Cargar y limpiar datos
9 val dfRaw = spark.read.option("header", "true").option("inferSchema", "true").csv("IGP_IonosferaRadarDigisonda_2013-2026
    ↳ _Dataset.csv")
10 val dfClean = dfRaw.na.drop(Seq("FREC_MAX_F2", "ALT_F", "ALT_MAX_DENS"))
11
12 // 2. Crear variable objetivo (Target) y quitar la original para evitar fugas de datos
13 val dfClass = dfClean.withColumn("NIVEL_ION",
14   when(col("FREC_MAX_F2") < 5.62, "Baja")
15   .when(col("FREC_MAX_F2") > 9.10, "Alta")
16   .otherwise("Normal")
17 ).drop("FREC_MAX_F2")
18
19 // 3. Preparar Features
20 val labelIndexer = new StringIndexer().setInputCol("NIVEL_ION").setOutputCol("label")
21 val assembler = new VectorAssembler().setInputCols(Array("HORA_UTC", "ANIO", "ALT_F", "ALT_MAX_DENS")).setOutputCol("
    ↳ rawFeatures")
22 val scaler = new MinMaxScaler().setInputCol("rawFeatures").setOutputCol("features")
23
```

```

24 // 4. División de datos
25 val Array(trainingData, testData) = dfClass.randomSplit(Array(0.8, 0.2), seed = 1234)
26
27 // 5. Instanciar Modelos
28 val dt = new DecisionTreeClassifier().setLabelCol("label").setFeaturesCol("features")
29 val rf = new RandomForestClassifier().setLabelCol("label").setFeaturesCol("features")
30 val lr = new LogisticRegression().setLabelCol("label").setFeaturesCol("features").setMaxIter(10)
31 val nb = new NaiveBayes().setLabelCol("label").setFeaturesCol("rawFeatures")
32
33 // 6. Pipelines
34 val pipeDT = new Pipeline().setStages(Array(labelIndexer, assembler, scaler, dt))
35 val pipeRF = new Pipeline().setStages(Array(labelIndexer, assembler, scaler, rf))
36 val pipeLR = new Pipeline().setStages(Array(labelIndexer, assembler, scaler, lr))
37 val pipeNB = new Pipeline().setStages(Array(labelIndexer, assembler, nb))
38
39 // 7. Grid-Search y Cross Validation
40 val evaluator = new MulticlassClassificationEvaluator().setLabelCol("label").setPredictionCol("prediction")
41
42 // Grillas de parámetros (reducidas para ejecución rápida)
43 val gridDT = new ParamGridBuilder().addGrid(dt.maxDepth, Array(5, 10)).build()
44 val gridRF = new ParamGridBuilder().addGrid(rf.numTrees, Array(10, 20)).build()
45 val gridLR = new ParamGridBuilder().addGrid(lr.regParam, Array(0.1, 0.01)).build()
46 val gridNB = new ParamGridBuilder().addGrid(nb.smoothing, Array(1.0)).build()
47
48 val cvDT = new CrossValidator().setEstimator(pipeDT).setEvaluator(evaluator.setMetricName("accuracy")).setEstimatorParamMaps(
49   ↳ gridDT).setNumFolds(3)
49 val cvRF = new CrossValidator().setEstimator(pipeRF).setEvaluator(evaluator.setMetricName("accuracy")).setEstimatorParamMaps(
50   ↳ gridRF).setNumFolds(3)
50 val cvLR = new CrossValidator().setEstimator(pipeLR).setEvaluator(evaluator.setMetricName("accuracy")).setEstimatorParamMaps(
51   ↳ gridLR).setNumFolds(3)
51 val cvNB = new CrossValidator().setEstimator(pipeNB).setEvaluator(evaluator.setMetricName("accuracy")).setEstimatorParamMaps(
52   ↳ gridNB).setNumFolds(3)
52
53 // 8. Entrenamiento
54 println("Entrenando modelos de clasificación")
55 println("Entrenando DECISION TREE")
56 val modelDT = cvDT.fit(trainingData)
57 println("Entrenando RANDOM FOREST")
58 val modelRF = cvRF.fit(trainingData)
59 println("Entrenando REGRESION LOGISTICA")
60 val modelLR = cvLR.fit(trainingData)
61 println("Entrenando NAIVE BAYES")
62 val modelNB = cvNB.fit(trainingData)
63
64 // 9. Predicciones
65 val predDT = modelDT.transform(testData)
66 val predRF = modelRF.transform(testData)
67 val predLR = modelLR.transform(testData)
68 val predNB = modelNB.transform(testData)
69
70 // 10. Función para extraer todas las métricas
71 def getMetrics(predictions: org.apache.spark.sql.DataFrame, modelName: String): Unit = {
72   val acc = evaluator.setMetricName("accuracy").evaluate(predictions)
73   val prec = evaluator.setMetricName("weightedPrecision").evaluate(predictions)
74   val rec = evaluator.setMetricName("weightedRecall").evaluate(predictions)
75   val f1 = evaluator.setMetricName("f1").evaluate(predictions)
76   println(f"$modelName%-20s | $acc%.4f | $prec%.4f | $rec%.4f | $f1%.4f")
77 }
78
79 // 11. Imprimir Tabla de Resultados
80 println("\n=== TABLA DE MÉTRICAS: CLASIFICACIÓN ===")
81 println("Modelo | Accuracy | Precision | Recall | F1-Score")
82 println("-----")
83 getMetrics(predDT, "Decision Tree")
84 getMetrics(predRF, "Random Forest")
85 getMetrics(predLR, "Regresión Logística")
86 getMetrics(predNB, "Naive Bayes")
87 println("=====\n")
88
89 // Extraer historial de Regresión Logística
90 val lrModelFinalClass = modelLR.bestModel.asInstanceOf[org.apache.spark.ml.PipelineModel].stages.last.asInstanceOf[org.apache.
91   ↳ spark.ml.classification.LogisticRegressionModel]
91 val lossHistoryLRClass = lrModelFinalClass.summary.objectiveHistory
92
93 println("\n--- Listado de loss_clasificacion ---")
94 println("loss_clasificacion = [" + lossHistoryLRClass.mkString(", ") + "]")

```

7.3. Regresión sin XGBoost (regresion_completa-s-XGBoostRegressor.scala)

```

1 import org.apache.spark.sql.functions._
2 import org.apache.spark.ml.Pipeline
3 import org.apache.spark.ml.feature.{VectorAssembler, StandardScaler}

```

```

4 import org.apache.spark.ml.regression.{LinearRegression, RandomForestRegressor, GBRegressor}
5 import org.apache.spark.ml.evaluation.RegressionEvaluator
6 import org.apache.spark.ml.tuning.{ParamGridBuilder, CrossValidator}
7 import org.apache.spark.ml.PipelineModel
8
9 // 1. Cargar y limpiar datos
10 val dfRaw = spark.read.option("header", "true").option("inferSchema", "true").csv("IGP_IonosferaRadarDigisonda_2013-2026
    ↳ _Dataset.csv")
11 val dfClean = dfRaw.na.drop(Seq("FREC_MAX_F2", "ALT_F", "ALT_MAX_DENS"))
12
13 // 2. Preparar variable objetivo y features
14 val dfReg = dfClean.withColumnRenamed("FREC_MAX_F2", "label")
15 val assembler = new VectorAssembler().setInputCols(Array("HORA_UTC", "ANIO", "ALT_F", "ALT_MAX_DENS")).setOutputCol("
    ↳ rawFeatures")
16 val scaler = new StandardScaler().setInputCol("rawFeatures").setOutputCol("features").setWithStd(true).setWithMean(true)
17
18 // 3. División de datos
19 val Array(trainingData, testData) = dfReg.randomSplit(Array(0.7, 0.3), seed = 1234)
20
21 // 4. Instanciar Modelos (Nativos de Spark que no crashean en WSL2)
22 val lr = new LinearRegression().setLabelCol("label").setFeaturesCol("features")
23 val rfr = new RandomForestRegressor().setLabelCol("label").setFeaturesCol("features")
24 val gbt = new GBRegressor().setLabelCol("label").setFeaturesCol("features").setMaxIter(5)
25 // val xgb = new XGBoostRegressor().setLabelCol("label").setFeaturesCol("features") // Omitido por incompatibilidad de red en
    ↳ WSL2
26
27 // 5. Pipelines
28 val pipeLR = new Pipeline().setStages(Array(assembler, scaler, lr))
29 val pipeRFR = new Pipeline().setStages(Array(assembler, scaler, rfr))
30 val pipeGBT = new Pipeline().setStages(Array(assembler, scaler, gbt))
31
32 // 6. Evaluador y Grillas
33 val evaluatorReg = new RegressionEvaluator().setLabelCol("label").setPredictionCol("prediction")
34
35 val gridLR = new ParamGridBuilder().addGrid(lr.regParam, Array(0.1)).build()
36 val gridRFR = new ParamGridBuilder().addGrid(rfr.numTrees, Array(10)).build()
37 val gridGBT = new ParamGridBuilder().addGrid(gbt.maxDepth, Array(3)).build()
38
39 val cvLR = new CrossValidator().setEstimator(pipeLR).setEvaluator(evaluatorReg.setMetricName("rmse")).setEstimatorParamMaps(
    ↳ gridLR).setNumFolds(2)
40 val cvRFR = new CrossValidator().setEstimator(pipeRFR).setEvaluator(evaluatorReg.setMetricName("rmse")).setEstimatorParamMaps(
    ↳ gridRFR).setNumFolds(2)
41 val cvGBT = new CrossValidator().setEstimator(pipeGBT).setEvaluator(evaluatorReg.setMetricName("rmse")).setEstimatorParamMaps(
    ↳ gridGBT).setNumFolds(2)
42
43 // 7. Entrenamiento Seguro
44 println("Entrenando Regresión Lineal...")
45 val modelLR = cvLR.fit(trainingData)
46
47 println("Entrenando Random Forest...")
48 val modelRFR = cvRFR.fit(trainingData)
49
50 println("Entrenando GBT...")
51 val modelGBT = cvGBT.fit(trainingData)
52
53 // 8. Predicciones seguras
54 val predLR = modelLR.transform(testData)
55 val predRFR = modelRFR.transform(testData)
56 val predGBT = modelGBT.transform(testData)
57
58 // 9. Función de Métricas
59 def getRegMetrics(predictions: org.apache.spark.sql.DataFrame, modelName: String): Unit = {
60     val rmse = evaluatorReg.setMetricName("rmse").evaluate(predictions)
61     val mse = evaluatorReg.setMetricName("mse").evaluate(predictions)
62     val mae = evaluatorReg.setMetricName("mae").evaluate(predictions)
63     val r2 = evaluatorReg.setMetricName("r2").evaluate(predictions)
64     println(f"$modelName %-20s | $rmse %.4f | $mse %.4f | $mae %.4f | $r2 %.4f")
65 }
66
67 // 10. Imprimir Tabla de Resultados Final
68 println("\n=== TABLA DE MÉTRICAS: REGRESIÓN ===")
69 println("Modelo          | RMSE   | MSE    | MAE    | R^2")
70 println("-----")
71 getRegMetrics(predLR, "Linear Regression")
72 getRegMetrics(predRFR, "Random Forest Reg.")
73 getRegMetrics(predGBT, "GBT Regressor")
74 println("XGBoost Regressor    | --- Omitido por limitación NAT en WSL2 ---")
75 println("=====n")
76
77 // 11. Extraer Pérdidas (Obligando al optimizador iterativo l-bfgs para evitar el 0.0)
78 val lrGrafico = new LinearRegression().setLabelCol("label").setFeaturesCol("features").setMaxIter(15).setSolver("l-bfgs")
79 val pipeGrafico = new Pipeline().setStages(Array(assembler, scaler, lrGrafico))
80 val modeloGrafico = pipeGrafico.fit(trainingData)
81 val historyGrafico = modeloGrafico.stages.last.asInstanceOf[org.apache.spark.ml.regression.LinearRegressionModel].summary.
    ↳ objectiveHistory

```

```

82 println("\n--- Lista de loss_regresion ---")
83 println("loss_regresion = [" + historyGrafico.mkString(", ") + "]\n")

```

7.4. Regresión con XGBoost (regresion_completa-c-XGBoostRegressor.scala)

```

1 import org.apache.spark.sql.functions._
2 import org.apache.spark.ml.Pipeline
3 import org.apache.spark.ml.feature.{VectorAssembler, StandardScaler}
4 import org.apache.spark.ml.regression.{LinearRegression, RandomForestRegressor, GBRegressor}
5 import org.apache.spark.ml.evaluation.RegressionEvaluator
6 import org.apache.spark.ml.tuning.{ParamGridBuilder, CrossValidator}
7 import org.apache.spark.ml.PipelineModel
8 import ml.dmlc.xgboost4j.scala.spark.XGBoostRegressor
9
10 // 1. Cargar y limpiar datos
11 val dfRaw = spark.read.option("header", "true").option("inferSchema", "true").csv("IGP-IonosferaRadarDigisonda_2013-2026
    ↳ _Dataset.csv")
12 val dfClean = dfRaw.na.drop(Seq("FREC_MAX_F2", "ALT_F", "ALT_MAX_DENS"))
13
14 // 2. Preparar variable objetivo y features
15 val dfReg = dfClean.withColumnRenamed("FREC_MAX_F2", "label")
16 val assembler = new VectorAssembler().setInputCols(Array("HORA_UTC", "ANIO", "ALT_F", "ALT_MAX_DENS")).setOutputCol("
    ↳ rawFeatures")
17 val scaler = new StandardScaler().setInputCol("rawFeatures").setOutputCol("features").setWithStd(true).setWithMean(true)
18
19 // 3. División de datos
20 val Array(trainingData, testData) = dfReg.randomSplit(Array(0.7, 0.3), seed = 1234)
21
22 // 4. Instanciar Modelos
23 val lr = new LinearRegression().setLabelCol("label").setFeaturesCol("features")
24 val rfr = new RandomForestRegressor().setLabelCol("label").setFeaturesCol("features")
25 val gbt = new GBRegressor().setLabelCol("label").setFeaturesCol("features").setMaxIter(5)
26 val xgb = new XGBoostRegressor().setLabelCol("label").setFeaturesCol("features").setNumRound(5).setNumWorkers(1)
27
28 // 5. Pipelines
29 val pipeLR = new Pipeline().setStages(Array(assembler, scaler, lr))
30 val pipeRFR = new Pipeline().setStages(Array(assembler, scaler, rfr))
31 val pipeGBT = new Pipeline().setStages(Array(assembler, scaler, gbt))
32 val pipeXGB = new Pipeline().setStages(Array(assembler, scaler, xgb))
33
34 // 6. Evaluador y Grillas
35 val evaluatorReg = new RegressionEvaluator().setLabelCol("label").setPredictionCol("prediction")
36
37 val gridLR = new ParamGridBuilder().addGrid(lr.regParam, Array(0.1)).build()
38 val gridRFR = new ParamGridBuilder().addGrid(rfr.numTrees, Array(10)).build()
39 val gridGBT = new ParamGridBuilder().addGrid(gbt.maxDepth, Array(3)).build()
40
41 val cvLR = new CrossValidator().setEstimator(pipeLR).setEvaluator(evaluatorReg.setMetricName("rmse")).setEstimatorParamMaps(
    ↳ gridLR).setNumFolds(2)
42 val cvRFR = new CrossValidator().setEstimator(pipeRFR).setEvaluator(evaluatorReg.setMetricName("rmse")).setEstimatorParamMaps(
    ↳ gridRFR).setNumFolds(2)
43 val cvGBT = new CrossValidator().setEstimator(pipeGBT).setEvaluator(evaluatorReg.setMetricName("rmse")).setEstimatorParamMaps(
    ↳ gridGBT).setNumFolds(2)
44
45 // 7. Entrenamiento Seguro
46 println("Entrenando Regresión Lineal...")
47 val modelLR = cvLR.fit(trainingData)
48
49 println("Entrenando Random Forest...")
50 val modelRFR = cvRFR.fit(trainingData)
51
52 println("Entrenando GBT...")
53 val modelGBT = cvGBT.fit(trainingData)
54
55 // [BLOQUE TRY-CATCH PARA PROTEGER EL SCRIPT DE WSL2]
56 var predXGB: org.apache.spark.sql.DataFrame = null
57 var xgbExitoso = false
58
59 println("Entrenando XGBoost...")
60 try {
61     val modelXGB = pipeXGB.fit(trainingData)
62     predXGB = modelXGB.transform(testData)
63     xgbExitoso = true
64 } catch {
65     case e: Throwable =>
66         println("\n[!] AVISO: XGBoost fue bloqueado por la red de WSL2 (RabitTracker).")
67         println("[!] El script continuará automáticamente con los demás modelos.\n")
68 }
69
70 // 8. Predicciones seguras
71 val predLR = modelLR.transform(testData)

```

```

72 val predRFR = modelRFR.transform(testData)
73 val predGBT = modelGBT.transform(testData)
74
75 // 9. Función de Métricas
76 def getRegMetrics(predictions: org.apache.spark.sql.DataFrame, modelName: String): Unit = {
77   val rmse = evaluatorReg.setMetricName("rmse").evaluate(predictions)
78   val mse = evaluatorReg.setMetricName("mse").evaluate(predictions)
79   val mae = evaluatorReg.setMetricName("mae").evaluate(predictions)
80   val r2 = evaluatorReg.setMetricName("r2").evaluate(predictions)
81   println(f"$modelName %-20s | $rmse%.4f | $mse%.4f | $mae%.4f | $r2%.4f")
82 }
83
84 // 10. Imprimir Tabla de Resultados Final
85 println("\n=== TABLA DE MÉTRICAS: REGRESIÓN ===")
86 println("Modelo          | RMSE    | MSE     | MAE     | R^2")
87 println("-----")
88 getRegMetrics(predLR, "Linear Regression")
89 getRegMetrics(predRFR, "Random Forest Reg.")
90 getRegMetrics(predGBT, "GBT Regressor")
91
92 if(xgbExitoso) {
93   getRegMetrics(predXGB, "XGBoost Regressor")
94 } else {
95   println("XGBoost Regressor    | --- Omitido por bloqueo de red en WSL2 ---")
96 }
97 println("=====\n")
98
99 // 11. Extraer Pérdidas de Regresión Lineal para graficar
100 val lrModelFinalReg = modelLR.bestModel.asInstanceOf[org.apache.spark.ml.regression.
    ↳ LinearRegressionModel]
101 val lossHistoryLRReg = lrModelFinalReg.summary.objectiveHistory
102
103 println("\n--- Listado de loss_regresion ---")
104 println("loss_regresion = [" + lossHistoryLRReg.mkString(", ") + "]")

```

7.5. Generación de gráficas de pérdidas (graficar_perdidas.py)

```

import matplotlib.pyplot as plt

# 1. PEGA AQUÍ LAS LISTAS QUE TE ARROJÓ SCALA EN LA CONSOLA
loss_clasificacion = [1.0371916103595906, 0.8892437094603101, 0.866203591954148, 0.8597180458716372, 0.856679856159765,
    ↳ 0.8557007568144392, 0.8555525923151555, 0.8555270169475461, 0.8555260231039171, 0.8555259731842708, 0.8555259662609936]
    ↳ # Reemplazar con tus valores
loss_regresion = [0.49999748927922333, 0.436759981611607, 0.31605871451812534, 0.3137551766030895, 0.3122770070808429,
    ↳ 0.3121787321778962, 0.3121480124157871, 0.3121480084787035, 0.3121480084201449, 0.312148008419911] # Reemplazar con tus
    ↳ valores

# Crear la figura con dos subgráficos
plt.figure(figsize=(12, 5))

# Gráfico 1: Clasificación (Regresión Logística)
plt.subplot(1, 2, 1)
plt.plot(loss_clasificacion, marker='o', linestyle='-', color='b', linewidth=2)
plt.title('Pérdidas de Entrenamiento - Clasificación\n(Regresión Logística Multiclase)', fontsize=12)
plt.xlabel('Iteraciones', fontsize=10)
plt.ylabel('Pérdida (Objective)', fontsize=10)
plt.grid(True, linestyle='--', alpha=0.7)

# Gráfico 2: Regresión (Regresión Lineal)
plt.subplot(1, 2, 2)
plt.plot(loss_regresion, marker='s', linestyle='-', color='r', linewidth=2)
plt.title('Pérdidas de Entrenamiento - Regresión\n(Regresión Lineal)', fontsize=12)
plt.xlabel('Iteraciones', fontsize=10)
plt.ylabel('Pérdida (Objective)', fontsize=10)
plt.grid(True, linestyle='--', alpha=0.7)

# Ajustar layout y guardar la imagen de alta calidad para el informe
plt.tight_layout()
plt.savefig('graficos_perdidas_modelos.png', dpi=300)
print("Los gráficos se han generado correctamente en 'graficos_perdidas_modelos.png'")

```

8. Anexo: Documentación

El repositorio del proyecto (Figura 3 y 4) contiene, además del código, el dataset, el diccionario de datos (DiccionarioDatosDigisonda.xlsx), los metadatos oficiales del IGP y el archivo `requirements.txt` con las dependencias de Python. La estructura del repositorio es:

```
PC4/
|-- Comandos/README.md                # Comandos y resultados
|-- IGP_IonosferaRadarDigisonda_2013-2026_Dataset.csv
|-- DiccionarioDatosDigisonda.xlsx
|-- IGP_IonosferaRadarDigisonda_2013-2026_Metadatos.docx
|-- requirements.txt
|-- summary.py
|-- Img/
|   |-- Clasificacion.png
|   |-- Regresion-s-XGBoost.png
|-- src/
|   |-- clasificacion_completa.scala
|   |-- regresion_completa-s-XGBoostRegressor.scala
|   |-- regresion_completa-c-XGBoostRegressor copy.scala
|   |-- graficar_perdidas.py
|   |-- graficos_perdidas_modelos.png
```

Referencias

- [1] Apache Software Foundation. Mllib: Main guide - spark machine learning library, 2024. <https://spark.apache.org/docs/latest/ml-guide.html>.
- [2] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD*, 2016.
- [3] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, 2 edition, 2009.
- [4] Instituto Geofísico del Perú (IGP). Datos de la ionósfera obtenidos con el radar digisonda. Plataforma Nacional de Datos Abiertos (PNDA), 2026. <https://www.datosabiertos.gob.pe/dataset/datos-de-la-ionosfera-obtenidos-con-el-radar-digisonda-instituto-geofisico-del-peru-igp>.
- [5] Henry Rishbeth and Owen K. Garriott. *Introduction to Ionospheric Physics*. Academic Press, 1969.